

Caso semestral

Sigla	Nombre Asignatura
DSY1106	DESARROLLO FULLSTACK III

Caso Semestral: "Donaton – plataforma inteligente para la gestión y coordinación de ayuda humanitaria"

En situaciones de emergencia, desastres naturales o crisis sociales, coordinar de forma eficiente la ayuda es un factor principal para asegurar que los recursos lleguen de manera oportuna a quienes más lo necesitan. Lamentablemente en muchos casos la gestión de estas donaciones se realiza mediante procesos manuales, hojas de cálculo o sistemas aislados generando dificultades en la coordinación de las diferentes instituciones, organizaciones o personas que participan.

Donaton es una organización especializada a la gestión de ayuda humanitaria y coordinación de donaciones, que trabaja de manera activa con diversas instituciones para llegar a las comunidades afectadas en las diversas situaciones de emergencia que se presentan en el país.

La organización ha generado un fuerte impacto de responsabilidad social a través de voluntarios, centros de acopio logrando recibir diferentes tipos de donaciones, incluyendo ropa, alimento, insumos médicos, insumos de higiene y otro tipo de elementos para satisfacer las necesidades básicas de los damnificados.

El crecimiento de la organización ha generado nuevos desafíos operacionales, en donde las herramientas tecnológicas pueden ayudar a mejorar los procesos logísticos y administrativos.

Para resolver estos problemas, Donaton quiere desarrollar una solución basada en microservicios, con una capa frontend moderna y flexible, y un backend escalable y seguro, permitiendo centralizar la gestión de donaciones, optimizar los procesos y mejorar la coordinación entre los distintos involucrados, proporcionando mayor transparencia y uso de los recursos.

El desarrollo de este sistema se llevará a cabo en **tres etapas**, alineadas con las evaluaciones parciales del curso. Finalmente, en el **Examen Final Transversal**, los/as estudiantes consolidarán su solución integrando todos los módulos desarrollados en un sistema funcional.

Sección 1: Diseño de Arquitectura y Patrones de Microservicios (Parcial 1)

Contexto

Donaton tiene un alcance regional coordinando donaciones provenientes de distintas entidades:

- Personas que realizan donaciones de manera individual
- Empresas que realizan donaciones en grandes volúmenes
- Municipalidades que analizan, documentan y gestionan las necesidades de la comunidad
- Centros de acopio en donde se almacena temporalmente la ayuda
- Equipos logísticos que se encargan del transporte y distribución de donaciones.

En el último tiempo gran parte de las donaciones realizadas se han gestionado de forma directa, mediante redes sociales, comunicaciones informales, generando problemas importantes de coordinación. Dichas limitaciones concluyen en donaciones que llegan a damnificados que no lo requieren, o se producen duplicaciones de recursos mientras en otros lugares quedan desabastecidos.

Para mejorar esta situación y transparentar el flujo completo desde que alguien efectúa una donación hasta su recepción final, Donaton ha decidido desarrollar una plataforma tecnológica que permita gestionar de manera centralizada las necesidades, donaciones y logística de distribución

La solución debe contemplar tres módulos principales:

- **Gestión de donaciones:** Permite administrar las donaciones recibidas por la organización detallando recurso, cantidad, origen, fecha y centro de acopio asignado.
- **Gestión de logística y distribución:** Administrar los centros de acopio, controlar inventario, planificar envíos, asignando el transporte y realizar seguimiento del proceso hasta la entrega final.
- **Gestión de necesidades en terreno:** Permitir que usuarios externos puedan reportar las necesidades específicas, incluyendo los recursos necesitados, cantidad y ubicación geográfica.

Requerimientos Técnicos

Los/as estudiantes deberán diseñar una **arquitectura de microservicios escalable**, aplicando **patrones de diseño y arquetipos arquitectónicos** que permitan la modularización del sistema. Para ello, deberán:

- **Definir los microservicios clave**, asegurando separación de responsabilidades y escalabilidad.

- Diseñar una **API Gateway** que gestione la comunicación entre microservicios y el frontend.
- Implementar patrones como **Repository Pattern** para la persistencia de datos, **Factory Method** para la creación de instancias y **Circuit Breaker** para manejar fallos en la comunicación entre servicios.
- Asegurar que los servicios sean **escalables y desacoplados**, permitiendo futuras mejoras sin afectar el funcionamiento del sistema.
- Documentar las decisiones arquitectónicas y justificar la selección de patrones.

Al finalizar esta etapa, los equipos deberán presentar un informe con la propuesta de arquitectura, un diagrama detallado de los microservicios y una justificación de los patrones seleccionados.

Sección 2: Desarrollo de Componentes Frontend y Backend (Parcial 2)

Contexto

Después de definir la arquitectura del sistema, Donaton busca desarrollar una primera versión funcional que permita validar la propuesta tecnológica. La empresa requiere un sistema que cuente con una **interfaz de usuario intuitiva y responsiva**. El backend debe ser capaz de manejar múltiples solicitudes concurrentes sin comprometer el rendimiento.

Requerimientos Técnicos

Los/as estudiantes deberán desarrollar la solución con los siguientes elementos clave:

- **Frontend:** Implementar una interfaz construida con un framework moderno (React, Angular o Vue.js), asegurando que la comunicación con el backend se realice vía API REST. Los componentes frontend deberán ser empaquetados como **módulos NPM** reutilizables.
- **Backend:** Construir al menos tres componentes backend utilizando **arquetipos Maven personalizados**:
 - Un **Backend For Frontend (BFF)** para gestionar la interacción entre el frontend y los microservicios.
 - Dos **microservicios independientes**, conectados a bases de datos mediante JPA.
- **Conexión con Bases de Datos:** Utilizar **JPA y entidades**, asegurando la persistencia de datos. También se podrán utilizar procedimientos almacenados (SPs) para optimizar operaciones.
- **Versionamiento del Código:** Todos los componentes deberán ser versionados en **Git**, utilizando estrategias de branching como Git Flow o GitHub Flow para facilitar el trabajo colaborativo.

- **Implementación de Patrones de Diseño:** Aplicar patrones adecuados para mejorar la organización y mantenibilidad del código, asegurando que los servicios sean modulares y fácilmente extensibles.

Como resultado de esta etapa, los/as estudiantes deberán entregar la primera versión funcional del sistema, acompañada de una presentación donde expliquen las decisiones técnicas, la implementación de los componentes y la integración entre frontend y backend.

Sección 3: Integración, Pruebas Unitarias y Presentación Final (Parcial 3)

Contexto

Con el sistema en funcionamiento, Donaton quiere asegurar que la solución sea robusta y confiable antes de su implementación definitiva. Para ello, se requiere que el código desarrollado cumpla con **estándares de calidad**, aplicando **pruebas unitarias** y validando la **cobertura del código** antes del lanzamiento.

En esta última fase, la empresa evaluará cómo se integran los diferentes módulos del sistema, garantizando que la plataforma sea escalable y esté lista para su despliegue en producción.

Requerimientos Técnicos

En esta etapa, los/as estudiantes deberán:

- **Realizar Pruebas Unitarias:** Implementar pruebas para cada uno de los componentes del sistema, asegurando una cobertura mínima del **60% del código**. La validación de cobertura se realizará utilizando herramientas como **SonarQube**.
- **Refactorización y Mejora del Código:** Revisar el código desarrollado, identificar oportunidades de optimización y aplicar mejoras siguiendo buenas prácticas de desarrollo.
- **Despliegue y Ejecución de Pruebas:** Integrar las pruebas unitarias en un pipeline de **Integración Continua**, asegurando que se ejecuten automáticamente en cada actualización del código.
- **Documentación Final:** Completar la documentación del sistema, incluyendo diagramas actualizados, descripción de la arquitectura final y detalles sobre la implementación de pruebas.

Presentación Final

Cada equipo presentará su solución ante el/la docente, abordando los siguientes puntos clave:

- **Explicación de la arquitectura del sistema y decisiones técnicas.**

- **Demostración del funcionamiento del software, mostrando la integración entre frontend y backend.**
- **Estrategia de pruebas implementada y validación de cobertura.**
- **Reflexión sobre los desafíos enfrentados y aprendizajes adquiridos.**

Esta presentación servirá como cierre del curso, permitiendo evaluar de manera integral la capacidad de los/as estudiantes para diseñar, desarrollar, integrar y validar una solución de software basada en **microservicios, patrones de diseño y pruebas automatizadas**.